FULL PLAN TRIỂN KHAI
HỆ THỐNG PHÂN TÍCH ĐÓNG GÓP DEV TỪ GITHUB

Tài liệu thực thi cho công cụ nội bộ của công ty

Stack chốt: React + Python FastAPI + Laragon MySQL

Mục tiêu: tạo hệ thống thu thập dữ liệu GitHub, phân tích commit/diff/PR, chấm điểm đóng góp minh bạch và hiển thị trên website nội bộ.

Tên dự án	Engineering Contribution Analytics
Phạm vi V1	1 tổ chức GitHub, nhiều repo nội bộ, dashboard team/dev, scoring bản đầu, phân tích AI commit/PR
Nguyên tắc	Không dùng số commit đơn thuần làm KPI; phải có giải thích, có bằng chứng, có khả năng kiểm tra ngược
Định hướng triển khai	Triển khai nội bộ, ít hạ tầng phụ, không Docker, không CI/CD bắt buộc ở giai đoạn đầu

MỤC LỤC TÓM TẮT

- 1. Tóm tắt điều hành
- 2. Bối cảnh và bài toán
- 3. Mục tiêu, phạm vi và nguyên tắc công bằng
- 4. Kiến trúc tổng thể hệ thống
- 5. Stack công nghệ chốt
- 6. Thiết kế dữ liệu và scoring engine
- 7. Thiết kế AI và cách dùng model có sẵn
- 8. Thiết kế backend, frontend và tích hợp GitHub
- 9. Kế hoạch triển khai theo giai đoạn
- 10. Vận hành nội bộ, bảo mật và rủi ro
- 11. Tiêu chí nghiệm thu và roadmap mở rộng

Ghi chú: tài liệu này ưu tiên tính thực thi. Những gì không cần thiết cho công cụ nội bộ sẽ bị loại bỏ.

1. TÓM TẮT ĐIỀU HÀNH

Dự án này nhằm xây một công cụ nội bộ giúp thống kê và phân tích mức độ đóng góp của lập trình viên dựa trên dữ liệu GitHub. Hệ thống phải đọc commit, diff, pull request, review và các tín hiệu liên quan; sau đó tổng hợp thành dashboard, bảng xếp hạng và bản giải thích chi tiết để mọi người có thể xem trực tiếp trên website nội bộ.

Mục tiêu đúng không phải là tạo một “máy chấm KPI tuyệt đối”, mà là tạo một hệ thống phân tích đóng góp đủ công bằng để hỗ trợ quản lý team, soi được thói quen commit, thấy được khối lượng công việc đã giao hàng, và giảm tranh cãi cảm tính.

Kết luận triển khai: Dự án khả thi về kỹ thuật, phù hợp để làm nội bộ bằng React + FastAPI + MySQL trên Laragon. Không cần train model riêng ở giai đoạn đầu. Nên dùng model có sẵn qua API để phân tích commit/diff theo structured output.

2. BỐI CẢNH VÀ BÀI TOÁN

Thực tế trong team dev thường có hành vi commit rất khác nhau: có người gom nhiều tính năng rồi commit một lần; có người commit rất vụn; có người viết commit message cẩn thận; có người viết mơ hồ; có người để AI sinh message. Nếu lấy số commit hoặc số dòng code làm thước đo trực tiếp thì kết quả sẽ méo và gây phản ứng.

Các vấn đề hệ thống cần giải quyết:

- Chuẩn hóa các kiểu commit khác nhau để không thưởng sai người commit nhiều và không phạt oan người commit dồn.
- Đọc được nội dung thay đổi thực tế từ diff/PR thay vì chỉ nhìn commit message.
- Nhóm nhiều commit nhỏ thành một cụm công việc hợp lý để đánh giá theo work item.
- Phân loại đóng góp: feature, bugfix, refactor, test, docs, config, hotfix, review.
- Chấm điểm có giải thích, có link bằng chứng, có mức độ tin cậy.
- Hiển thị toàn bộ lên website nội bộ để team theo dõi minh bạch.

Những điều hệ thống không nên hứa hẹn:

- Không tuyên bố đánh giá năng lực tuyệt đối chỉ từ GitHub.
- Không dùng raw commit count làm thước đo chính.
- Không dùng raw LOC làm thước đo chính.
- Không để AI tự quyết định điểm cuối cùng mà không có rule engine và evidence.

3. MỤC TIÊU, PHẠM VI VÀ NGUYÊN TẮC CÔNG BẰNG

3.1 Mục tiêu chính

- Tự động đồng bộ dữ liệu GitHub về hệ thống nội bộ.
- Phân tích commit, diff, PR, review để sinh thống kê và insight.
- Tạo score tổng hợp có giải thích minh bạch.
- Cho phép xem theo team, theo repo, theo dev, theo mốc thời gian.
- Hỗ trợ lead/manager có thêm dữ liệu quản trị, không thay thế hoàn toàn đánh giá con người.

3.2 Phạm vi V1

- Đọc dữ liệu từ GitHub organization hoặc một nhóm repository được chỉ định.
- Hỗ trợ đồng bộ commit, commit diff, pull request, review, issue link cơ bản, branch, merge status.
- Xây dashboard nội bộ gồm trang tổng quan team, trang cá nhân dev, trang chi tiết work item, commit, PR.
- Có scoring engine bản đầu và AI analysis commit/PR.
- Có cấu hình loại trừ bot, loại trừ repo, ánh xạ alias email/username của dev.

3.3 Ngoài phạm vi V1

- Không tích hợp Jira, Slack, GitLab hoặc hệ thống HR ở giai đoạn đầu.
- Không làm mobile app.
- Không train model AI riêng.
- Không áp dụng tự động score làm căn cứ thưởng phạt chính thức ngay từ đầu.

3.4 Nguyên tắc công bằng bắt buộc

Nguyên tắc	Nội dung
Nguyên tắc 1	Chấm theo work item, không chấm theo raw commit.
Nguyên tắc 2	Commit message chỉ là tín hiệu phụ; diff và PR mới là nguồn chính.
Nguyên tắc 3	Review contribution phải có trọng số riêng để senior không bị thiệt.
Nguyên tắc 4	Score nào cũng phải giải thích được bằng evidence.
Nguyên tắc 5	Mọi công thức phải điều chỉnh được bằng cấu hình admin.

4. KIẾN TRÚC TỔNG THỂ HỆ THỐNG

Vì đây là công cụ nội bộ, kiến trúc cần ưu tiên ổn định, dễ vận hành và dễ sửa hơn là tối ưu quá mức. Không cần microservices. Chọn kiến trúc modular monolith là hợp lý nhất ở giai đoạn đầu.

4.1 Sơ đồ luồng tổng thể

```
[GitHub Webhook / GitHub API]
          ↓
[FastAPI Ingestion Layer]
          ↓
[MySQL Raw Tables + Normalized Tables]
          ↓
[Analysis Worker]
  - group commits thành work item
  - gọi AI để classify/summarize
  - tính metrics và score
          ↓
[FastAPI Application API]
          ↓
[React Internal Dashboard]
```

4.2 Kiến trúc thành phần

Thành phần	Vai trò
Frontend Web	React hiển thị dashboard, bảng xếp hạng, trang chi tiết dev/PR/work item, bộ lọc theo thời gian/repo/team.
API Backend	FastAPI cung cấp REST API cho UI, auth nội bộ, bộ lọc thống kê, trang admin cấu hình trọng số.
Ingestion Module	Đồng bộ dữ liệu từ GitHub theo webhook và job quét định kỳ.
Analysis Worker	Phân tích diff/PR, gom commit, gọi AI, tính score, ghi kết quả.
Scoring Engine	Rule engine quyết định điểm cuối cùng từ nhiều tín hiệu đã chuẩn hóa.
Database	MySQL trong Laragon lưu dữ liệu nghiệp vụ, log, cấu hình và snapshot điểm.

5. STACK CÔNG NGHỆ CHỐT

5.1 Stack bắt buộc theo yêu cầu

Lớp	Công nghệ	Lý do chọn

Lớp	Công nghệ	Lý do chọn
Frontend	React + TypeScript + Vite	Nhanh, gọn, phù hợp dashboard nội bộ, build đơn giản.
Backend	Python 3.11+ + FastAPI	Nhanh cho API, mạnh về data processing và AI integration.
Database	MySQL trên Laragon	Phù hợp môi trường nội bộ hiện có, dễ dựng và quản trị.

5.2 Stack bổ sung khuyến nghị

Nhóm	Công nghệ	Bắt buộc	Ghi chú
UI component	Ant Design	Nên dùng	Làm dashboard, table, filter, modal rất nhanh.
HTTP client FE	Axios	Có	Gọi API rõ ràng, dễ gắn interceptor auth.
State/query FE	@tanstack/react-query	Nên dùng	Cache và fetch dữ liệu thống kê tốt.
Chart FE	ECharts hoặc Recharts	Nên dùng	Biểu đồ team/repo/time series.
ORM BE	SQLAlchemy	Có	Tách mô hình dữ liệu sạch.
Migration	Alembic	Có	Quản lý schema MySQL bài bản.
Validation	Pydantic	Có	Schema request/response rõ ràng.
Server	Uvicorn	Có	Chạy FastAPI nhẹ, đủ cho nội bộ.
Background job	APScheduler + worker Python riêng	Có	Tránh phải kéo Redis/Celery ngay từ đầu.
Auth nội bộ	Local auth + role table	Có	Đơn giản nhất cho V1; có thể thêm SSO sau.
AI API	OpenAI Responses API	Có	Dùng model có sẵn, không train riêng.
Logging	Python logging + file log	Có	Đủ cho nội bộ; có thể thêm Sentry sau.

Khuyến nghị thực tế: Không kéo Redis, Celery, Kafka hay vector database vào V1 nếu chưa thật cần. Công cụ nội bộ nên giữ gọn: FastAPI + MySQL + worker Python riêng là đủ để chạy thật.

6. TRIỂN KHAI NỘI BỘ KHÔNG DOCKER

Do yêu cầu không dùng Docker/CI-CD ở giai đoạn đầu, phương án triển khai hợp lý là chạy trên máy nội bộ hoặc Windows Server trong mạng công ty.

6.1 Phương án triển khai đề xuất

- Laragon chạy MySQL và có thể tận dụng Nginx/Apache để serve static build của frontend.
- FastAPI chạy bằng Python virtual environment riêng.
- Worker phân tích chạy thành process Python riêng.
- Dùng NSSM hoặc Windows Task Scheduler để chạy API/worker như service nội bộ.
- Thư mục log, file cấu hình và backup đặt trên ổ đĩa nội bộ có phân quyền.

6.2 Cấu hình runtime khuyến nghị

Hạng mục	Giá trị
Máy chủ tối thiểu	4 vCPU, 8 GB RAM, SSD 100 GB
Khuyến nghị ổn hơn	8 vCPU, 16 GB RAM, SSD 200 GB
Hệ điều hành	Windows Server hoặc máy Windows nội bộ ổn định
Python	3.11 hoặc mới hơn
Node.js	20 LTS hoặc tương đương

7. TÍCH HỢP GITHUB

7.1 Cách kết nối

Có hai hướng: dùng Fine-grained Personal Access Token hoặc dùng GitHub App. Với công cụ nội bộ, cách đi khôn ngoan là chia làm hai bước:

- Giai đoạn dựng nhanh: dùng fine-grained token read-only để lấy dữ liệu và hoàn thành MVP.
- Giai đoạn ổn định hóa: chuyển sang GitHub App để quản trị quyền sạch hơn, dễ audit hơn, an toàn hơn.

7.2 Dữ liệu cần lấy

- Repositories
- Commits
- Commit diff / compare
- Pull requests
- Pull request commits
- Review và review comments
- Issue link cơ bản hoặc closing keyword
- Statuses / checks nếu cần dùng làm tín hiệu giao hàng
- Author identity: GitHub login, email, display name

7.3 Cơ chế đồng bộ

Cơ chế	Mô tả
Webhook	Nhận sự kiện push, pull_request, pull_request_review để cập nhật nhanh.
Periodic sync	Chạy job quét mỗi 15–30 phút để tránh miss webhook hoặc cập nhật dữ liệu cũ.
Backfill	Cho phép đồng bộ lịch sử 3–12 tháng ban đầu theo repo.
Retry	Nếu gọi API lỗi, đánh dấu retry theo hàng đợi trong DB.

8. THIẾT KẾ AI – ƯU TIÊN MODEL CÓ SẴN

Dự án không cần train AI riêng ở V1. Cách hiệu quả nhất là dùng model có sẵn qua API để đọc diff/PR và trả về dữ liệu có cấu trúc. AI chỉ là lớp phân tích; rule engine mới quyết định điểm cuối.

8.1 Model khuyến nghị

Hạng mục	Đề xuất	Cách dùng
Model chính	gpt-4.1-mini	Cân bằng tốt giữa chất lượng, tốc độ và chi phí; hợp bài toán đọc diff/PR, phân loại và sinh structured output.
Model rẻ hơn nếu cần	gpt-4.1-nano	Chỉ dùng cho tác vụ nhẹ như chuẩn hóa message hoặc pre-check đơn giản.
Embeddings tùy chọn V2	text-embedding-3-small	Dùng nếu sau này muốn grouping/semantic similarity tinh hơn.

8.2 AI phải làm những việc gì

- Tóm tắt commit hoặc PR theo ngôn ngữ dễ hiểu.
- Phân loại loại thay đổi: feature, bugfix, refactor, test, docs, config, chore, security, performance.
- Đối chiếu commit message với diff để chấm message_alignment_score.
- Ước lượng complexity, risk, test presence, coherence.
- Sinh lý do tăng/giảm điểm dưới dạng JSON schema rõ ràng.

8.3 AI không được làm gì

- Không tự quyết định final score mà không qua rule engine.
- Không tự suy diễn năng lực cá nhân ngoài dữ liệu có chứng cứ.
- Không chấm kỷ luật hoặc phán xét đạo đức làm việc.

8.4 Mẫu structured output đề xuất

```
{
  "change_type": "feature|bugfix|refactor|test|docs|config|chore|security|performance",
  "summary": "string",
  "complexity_score": 0-100,
  "risk_score": 0-100,
  "message_alignment_score": 0-100,
  "test_presence": true/false,
  "confidence": 0-1,
  "notes": ["..."]
}
```

Cần lưu cả model_version, prompt_version và schema_version để sau này audit được kết quả AI.

9. THIẾT KẾ SCORING ENGINE V1

Scoring engine phải minh bạch, chỉnh được bằng cấu hình và không được nghiêng quá mức về số commit. Cách an toàn nhất là dùng điểm tổng hợp từ nhiều lớp.

9.1 Ba lớp điểm

Lớp điểm	Ý nghĩa
Activity Score	Đo mức độ hoạt động: active days, PR count, review count. Không dùng làm lõi.
Contribution Quality Score	Đo mức độ có ý nghĩa của thay đổi: coherence, test, alignment, delivery, không-revert.
Impact Score	Đo mức đóng góp vào feature/bugfix/release/module quan trọng, dựa trên evidence.

9.2 Công thức V1 đề xuất

Contribution Score = 15% Activity + 50% Quality + 35% Impact

9.3 Các thành phần cụ thể

Nhóm	Feature đầu vào
Activity	active_days, merged_pr_count, review_count
Quality	work_item_coherence, message_alignment, test_presence, no_revert_ratio, meaningful_change_ratio
Impact	release_linked_changes, bugfix_weight, core_module_weight, delivery_score

9.4 Chống méo score do hành vi commit

- Gom commit gần nhau thành work item trước khi chấm.

- Commit quá nhỏ và quá dày sẽ bị giảm trọng số ở lớp hygiene, nhưng không triệt tiêu nếu thực tế giao hàng tốt.
- Commit quá lớn sẽ bị giảm coherence score và tăng review risk.
- Formatting-only, dependency lockfile, generated code phải có trọng số thấp hoặc bị loại trừ.
- Bot account bị loại khỏi bảng xếp hạng.

9.5 Output score bắt buộc phải có

- final_score
- confidence_score
- top_positive_reasons
- top_negative_reasons
- evidence_links
- score_breakdown

10. GOM COMMIT THÀNH WORK ITEM

Đây là điểm then chốt để giảm méo dữ liệu. Hệ thống không chấm từng commit độc lập, mà gom nhiều commit hợp lý thành một work item.

Tiêu chí gom đề xuất:

- Cùng author
- Gần nhau về thời gian
- Cùng branch hoặc cùng pull request
- Cùng ticket/issue keyword nếu có
- Có file/module overlap cao
- Có semantic similarity cao giữa summary/diff

Quy tắc thực thi V1:

- Ưu tiên gom theo PR nếu commit đã nằm trong PR.
- Nếu chưa có PR, gom theo branch + cửa sổ thời gian 4–24 giờ tùy cấu hình.
- Nếu commit quá lớn và đụng nhiều module không liên quan, giữ riêng work item để phạt coherence.

11. THIẾT KẾ DỮ LIỆU

11.1 Bảng chính

Bảng	Mục đích
developers	Thông tin dev, alias email/login, trạng thái active.
teams	Nhóm/team nội bộ.
repositories	Danh sách repo được theo dõi.
commits	Thông tin commit gốc.
commit_files	Danh sách file thay đổi theo commit.
pull_requests	Thông tin PR.

Bảng	Mục đích
pull_request_commits	Liên kết PR và commit.
reviews	Review và review comments tổng quan.
work_items	Cụm công việc sau khi gom commit.
work_item_commits	Liên kết work item và commit.
ai_commit_analysis	Kết quả AI phân tích commit/PR/work item.
score_snapshots	Điểm theo chu kỳ tính toán.
score_breakdowns	Chi tiết điểm thành phần.
app_configs	Trọng số, ngưỡng, danh sách loại trừ.
job_queue	Hàng đợi xử lý nền.
audit_logs	Nhật ký thao tác admin và pipeline.

11.2 Chỉ mục và khóa nên có

- Unique index cho repo_id + commit_sha
- Index cho author, repo, thời gian commit/push
- Index cho pull_request number + repo_id
- Index cho work_item_id và score period
- Index cho trạng thái job_queue

11.3 Dữ liệu cần chuẩn hóa

- Một dev có thể có nhiều email và nhiều username GitHub.
- Commit author và committer có thể khác nhau.
- Cần có bảng alias mapping để quy về một developer nội bộ.
- Cần cờ exclude_bot, exclude_generated_file, exclude_repo_from_ranking.

12. THIẾT KẾ BACKEND FASTAPI

12.1 Cấu trúc module đề xuất

```
backend/
  app/
    api/
    core/
    models/
    schemas/
    services/
    repositories/
    workers/
    scoring/
    github/
    ai/
    admin/
```

```
tests/
```

12.2 API chính cần có

Endpoint	Mục đích
GET /dashboard/overview	Tổng quan team theo bộ lọc thời gian/repo/team
GET /developers	Danh sách dev
GET /developers/{id}	Trang chi tiết một dev
GET /repositories	Danh sách repo
GET /work-items	Danh sách work item
GET /work-items/{id}	Chi tiết một work item
GET /pull-requests/{id}	Chi tiết PR
GET /scores/ranking	Bảng xếp hạng
POST /admin/sync/backfill	Chạy backfill dữ liệu
POST /admin/recalculate-score	Tính lại score
POST /github/webhook	Nhận webhook từ GitHub

12.3 Background jobs

- sync_repositories
- sync_commits
- sync_pull_requests
- build_work_items
- run_ai_analysis
- recalculate_scores
- cleanup_logs

12.4 Cách làm job nội bộ tối giản

Không cần Celery ở V1. Dùng một worker Python riêng, polling bảng job_queue trong MySQL. APScheduler chịu trách nhiệm đẩy job định kỳ vào queue. Cách này dễ hiểu, dễ debug, đủ cho công cụ nội bộ.

13. THIẾT KẾ FRONTEND REACT

13.1 Các màn hình chính

Màn hình	Nội dung
Login	Đăng nhập nội bộ theo role.
Dashboard tổng quan	Chỉ số toàn team, biểu đồ theo thời gian, top contributors.
Developer detail	Timeline đóng góp, score breakdown, work items, PR, review.

Màn hình	Nội dung
Repository detail	Hoạt động theo repo, module nóng, thay đổi nhiều.
Work item detail	Tóm tắt AI, commits liên quan, diff metadata, lý do ra điểm.
Admin settings	Cấu hình trọng số, ngưỡng, bot list, repo exclusions, lịch sync.

13.2 Thành phần UI quan trọng

- Bộ lọc theo thời gian, repo, team, developer
- Bảng ranking có sort và drill-down
- Biểu đồ active days / merged PR / score trend
- Badge hiển thị change_type
- Panel giải thích score
- Link out sang GitHub commit/PR gốc

14. BẢO MẬT VÀ PHÂN QUYỀN

14.1 Role tối thiểu

Role	Quyền
Admin	Quản trị hệ thống, cấu hình scoring, chạy sync/backfill, sửa alias.
Lead/Manager	Xem toàn bộ dashboard và chi tiết team.
Developer	Xem dashboard công khai nội bộ và trang cá nhân hoặc trang được cấp quyền.

14.2 Điểm bảo mật bắt buộc

- Token GitHub lưu ở file cấu hình bảo mật hoặc secret store nội bộ, không hard-code.
- Webhook phải kiểm tra chữ ký.
- API admin phải có role check.
- Ẩn dữ liệu repo nhạy cảm nếu có yêu cầu phân quyền theo team.
- Lưu audit log cho thao tác cấu hình score và dữ liệu backfill.

15. KẾ HOẠCH TRIỂN KHAI THEO 8 TUẦN

15.1 Lịch triển khai đề xuất

Tuần	Mục tiêu	Kết quả đầu ra	Người chịu trách nhiệm
Tuần 1	Khởi tạo dự án, chốt schema và kết nối GitHub	Repo FE/BE, env chạy được, MySQL schema draft, token/webhook test OK	Lead + BE
Tuần 2	Ingestion repo/commit/PR cơ bản	Đồng bộ dữ liệu thô về DB	BE
Tuần 3	Trang dashboard cơ bản + alias mapping	Xem được thống kê cơ bản theo dev/repo	FE + BE
Tuần 4	Work item grouping + rule engine nền	Commit được gom thành cụm công việc	BE
Tuần 5	Tích hợp AI phân tích commit/PR	Có summary, change type, complexity, alignment	BE
Tuần 6	Scoring V1 + admin settings	Có score breakdown, chỉnh trọng số được	BE + FE
Tuần 7	Tối ưu dashboard và test với dữ liệu thật	Bản beta nội bộ cho lead dùng	FE + BE + Lead
Tuần 8	Sửa lỗi, khóa phạm vi V1, hướng dẫn vận hành	Bản nghiệm thu nội bộ	Cả team

15.2 Nhân sự tối thiểu

- 01 Full-stack chính hoặc 01 BE + 01 FE
- 01 Lead/PM bán thời gian để chốt rule và nghiệm thu nghiệp vụ
- 01 người đại diện team dev để phản biện fairness của scoring

16. KẾ HOẠCH KIỂM THỬ

16.1 Nhóm test bắt buộc

- Test sync GitHub thành công và retry khi lỗi
- Test alias mapping đúng người
- Test grouping commit thành work item
- Test AI output đúng schema
- Test scoring đúng theo rule config
- Test phân quyền màn hình và API
- Test hiệu năng với dữ liệu lịch sử 3–6 tháng

16.2 Bộ dữ liệu kiểm thử nên chuẩn bị

- 1 dev commit rất vụn
- 1 dev commit dồn rất lớn
- 1 dev review nhiều nhưng code ít
- 1 dev chuyên test hoặc refactor
- 1 repo có generated files hoặc lockfile nặng
- 1 số commit message xấu để kiểm tra alignment

17. RỦI RO VÀ CÁCH KHÓA RỦI RO

Rủi ro	Biện pháp
Score gây tranh cãi	Công khai breakdown, cho admin chỉnh trọng số, không dùng làm KPI chính thức ngay.
AI phân loại sai	Lưu confidence, cho phép rerun, rule engine quyết định cuối.
Webhook miss sự kiện	Có periodic sync backfill ngắn.
Repo quá lớn	Backfill theo giai đoạn, batch processing, giới hạn diff size cho AI.
MySQL phình dữ liệu	Lưu patch/diff theo ngưỡng, archive dữ liệu cũ, index đúng.

18. TIÊU CHÍ NGHIỆM THU V1

- Đồng bộ được dữ liệu từ GitHub cho các repo đã chọn.
- Xem được dashboard team và trang cá nhân dev.
- Commit được gom thành work item hợp lý trong phần lớn trường hợp thực tế.
- AI trả ra summary và classification theo schema ổn định.
- Scoring engine cho ra điểm, breakdown và evidence link.
- Admin chỉnh được trọng số và chạy lại score.
- Hệ thống chạy ổn định nội bộ không cần Docker.

19. ROADMAP SAU V1

- Tích hợp issue tracker như Jira hoặc Linear để tăng độ chính xác của Impact Score.
- Thêm semantic grouping bằng embeddings khi dữ liệu đủ lớn.
- Thêm so sánh theo sprint hoặc release.
- Thêm review quality scoring.
- Thêm module export Excel/PDF cho manager.
- Thêm SSO nội bộ nếu công ty cần.

20. KẾT LUẬN CHỐT ĐỂ TRIỂN KHAI

Phương án đúng cho dự án này là làm một hệ thống contribution analytics nội bộ theo hướng modular monolith, React ở frontend, FastAPI ở backend, MySQL trên Laragon làm dữ liệu trung tâm, worker

Python riêng để xử lý nền, và dùng model có sẵn qua API để phân tích commit/PR. Không cần train model riêng, không cần kéo hạ tầng nặng.

Trong mọi giai đoạn, phải giữ một nguyên tắc: score chỉ có giá trị khi giải thích được bằng dữ liệu thật. Hệ thống này nên được dùng để tăng minh bạch và hỗ trợ quản lý team, không biến thành cây gậy KPI thô.

PHỤ LỤC A – CẤU TRÚC THƯ MỤC ĐỀ XUẤT

```
project-root/
  frontend/
  backend/
  scripts/
  docs/
  storage/
    logs/
    exports/
    raw_payloads/
  config/
```

PHỤ LỤC B – CẤU HÌNH MÔI TRƯỜNG TỐI THIỂU

```
GITHUB_TOKEN=...
GITHUB_WEBHOOK_SECRET=...
OPENAI_API_KEY=...
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=eng_analytics
DB_USER=...
DB_PASSWORD=...
```

Hết tài liệu. Bản này đủ để bắt đầu thiết kế chi tiết, chia việc và triển khai MVP nội bộ.